MSc Artificial Intelligence and Machine Learning - Final Project

University of Birmingham

School of Computer Science

Dynamic 3D Networks

Author:
Pascal Tohouri

ID No: 2512308

Course Lead:
Dr. Miqing Li

Instructor:
Dr. Rachid Anane

November 14, 2024

Contents

1 Geometric Networks

1.1 Introduction

There are many examples of high-performing deep neural networks (DNNs) [1][2][3][4]. DNNs have, however, provoked criticisms: they perform well [5][6][7], but researchers do not fully understand DNNs and cannot explain their outputs to high degrees of precision [5][8][9]. DNNs are also computationally expensive; they contain numerous trainable parameters that transform information [10][11][12]. Storing and training these parameters imposes large memory and GPU requirements [13][14][15][16].

A DNN is a computational graph $\mathbf{G}$, realised from the computational graph space $\mathbb{G}$. For example, if $\mathbf{G}$ is a multilayer perceptron with B biases and W weights and fixed activation functions:

$$\mathbf{G} \in \mathbb{G} \rightarrow \mathbf{x} \in \mathbb{R}^{B+W} \tag{1}$$

Searching for $\mathbf{G}$ in $\mathbb{G}$ equates to sampling $\mathbf{G}$ from a probability density function (PDF) over $\mathbb{G}$. The PDF maps from $\mathbb{G}$ to the relative likelihood a given graph is the *best-performing* graph. $\mathbb{R}^{B+W}$ is a high-dimensional space when the network has many parameters, so the PDF takes high-dimensional inputs.

Another way to describe $\mathbf{G}$ and $\mathbb{G}$ is with spatial coordinates. For example, let $\mathbb{G}$ occupy an Euclidean space:

$$\mathbf{G} \in \mathbb{G} \rightarrow \mathbf{G} \in \mathbb{R}^{D} \tag{2}$$

Now, rather than occupying a single point in the graph space, $\mathbf{G}$ is a physical system occupying multiple points. Let $\mathbf{x}^{(i)}$ denote node i's position in the D-dimensional space:

$$\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \ldots, x_D^{(i)}] \tag{3}$$

One defines $\mathbf{G}$ using the nodes' cartesian coordinates:

$$\mathbf{G} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \ldots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \ldots & x_D^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \ldots & x_D^{(N)} \end{bmatrix} \tag{4}$$

The connection likelihood can depend on the distance between nodes if the graph is not fully connected. Parameters, like weights and biases, can depend on spatial distances:

$$W = F_w(G) \tag{5}$$

$$B = F_b(G) \tag{6}$$

Networks in high-dimensional parameter spaces become well-defined in low-dimensional Euclidean spaces and intuitive to display. The PDF now maps from $\mathbb{R}^D$ to $\mathbb{R}_{\geq 0}$, thus allowing the removal of dimensionality issues like hyper-parameter counts that scale rapidly with the number of network parameters[1][17][18][19].

1.2 Dimensionality

Humans are most familiar with a space that seems locally like the 3D Euclidean space $\mathbf{x} = [x_1, x_2, x_3] \in \mathbb{R}^3$ [20][21][22]. A tradeoff exists wherein higher dimensions offer a richer space for constructing geometric networks and greater expressive power. Yet high dimensional spaces are increasingly less intuitive to visualise (Section B). So, the 3D Euclidean setting embeds the forthcoming architectures.

2 Model Dynamics

2.1 Optimal Graphs

Supposed an optimal graph $\mathbf{G}$ exists for any task. In 3D with parameter conditioning, the optimal graph is wholly defined by its nodes' coordinates. Thus, there exists an optimal number and spatial arrangement of nodes:

$$G^* = \begin{bmatrix} x_1^{*(1)} & x_2^{*(1)} & x_3^{*(1)} \\ x_1^{*(2)} & x_2^{*(2)} & x_3^{*(2)} \\ \vdots & \vdots & \vdots \\ x_1^{*(N^*)} & x_2^{(N^*)} & x_3^{*(N^*)} \end{bmatrix} \tag{7}$$

2.2 Probability Density Functions

Unlike in the MLP case, a PDF cannot map from the sample space $\mathbb{R}^3$ to the relative likelihood a given point contains the optimal network; a 3D network is not a single point in $\mathbb{R}^3$. Define a PDF on $\mathbb{R}^3$:

$$f : \mathbb{R}^3 \to \mathbb{R}_{\geq 0}. \tag{8}$$

For a given point $\mathbf{x}' \in \mathbb{R}^3$, the probability density $f(\mathbf{x}')$ corresponds to the probability a node arises at $\mathbf{x}'$. A geometric model seeks to realise $\mathbf{G}^*$ by establishing a dynamic relation

[1] "Hyper-parameter" refers to any information describing the PDF over $\mathbb{G}$

between the PDF f_t and its network realisation $\mathbf{G}_t$:

$$f \leftrightarrow \mathbf{G} \tag{9}$$

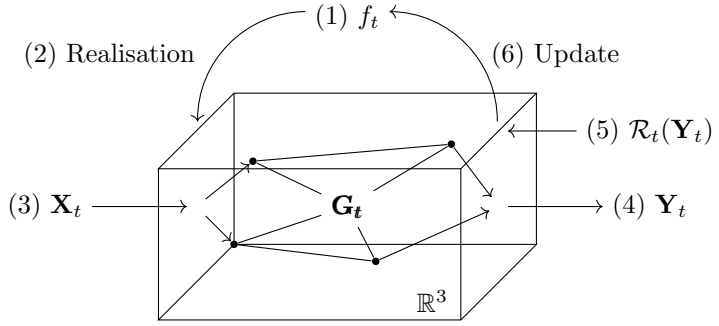


Figure 1: 3D Network Information Flow

To ensure task-optimal realisations:

1. Initialise PDF f_t over $\mathbb{R}^3$,
2. Collapse f_t to N hidden layer nodes with position-dependent weights and biases,
3. Forward Propagate input $\mathbf{X}_t$,
4. The network produces output $\mathbf{Y}_t$,
5. A pre-defined reward function returns $\mathcal{R}_t(\mathbf{Y}_t)$,
6. A reward-dependent transform updates the PDF: $f_{t+1} = T(f_t, \mathcal{R}_t)$.[2]

[2] $\mathcal{R}$ updates f at each time step and the continuous mapping $f \to \mathbf{G}$ may imply changes to $\mathbf{G}$ mid-operation.

3 Initialisation

3.1 Bounded Regions

The Euclidean 3-space is infinitely large. To ensure network nodes are visibly close, restrict the model to a bounded region[3] B:

$$B \subset \mathbb{R}^3 : \exists M \in (0, \infty) \tag{10}$$

$$\forall \mathbf{x} \in B, ||\mathbf{x}|| \leq M \tag{11}$$

Embedding $\mathbf{G}$ in B can simplify calculations by limiting edge length or node coordinate magnitudes. Appropriately choosing B can ease projection to the user's screen [23][24]. The uniform distribution can also act as an uninformative prior PDF[4]:

$$U(\mathbf{X}) = \begin{cases} \frac{1}{V} & \text{if } [x_1, x_2, x_3] \in B \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

$$V = \int_B 1 \, dV \tag{13}$$

3.2 Unbounded $\mathbb{R}^3$

A more general method is to let the graph occupy the unbounded $\mathbb{R}^3$ space but construct the realisation and updates to favour node proximity. One benefits from the non-zero probability of nodes with large norms when the PDF tends radially to zero [5]. Formally, for a focal point $\mathbf{x}'$, define a distribution $f(\mathbf{x})$ such that:

$$\lim_{\|\mathbf{x}' - \mathbf{x}\| \to \infty} f(\mathbf{x}) = 0 \tag{14}$$

3.3 Input-Output Nodes

Models tokenise input and output data in vector embeddings. Input and output token vectors $\mathbf{v}_I \in \mathbb{R}^{I \times 1}, \mathbf{v}_O \in \mathbb{R}^{O \times 1}$ pass through the input and output node positions $\mathbf{I}, \mathbf{O}$[6]:

$$\mathbf{I} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \\ x_2^{(2)} & x_2^{(2)} & x_3^{(2)} \\ \vdots & \vdots & \vdots \\ x_D^{(N_I)} & x_D^{(N_I)} & x_3^{(N_I)} \end{bmatrix}, \quad \mathbf{O} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \\ x_2^{(2)} & x_2^{(2)} & x_3^{(2)} \\ \vdots & \vdots & \vdots \\ x_D^{(N_O)} & x_D^{(N_O)} & x_3^{(N_O)} \end{bmatrix} \tag{15}$$

[3] Nodes arranged at very different scales are particularly difficult to visualise. Imagine small clusters of nodes in unit cubes separated by distances of millions of units!

[4] The uniform PDF must have a finite continuous support.

[5] In theory; in practice extremely distal PDF values underflow to zero.

[6] $\mathbf{I}$, $\mathbf{O}$ information may enter and exit through directed edges (to avoid output (input) data arriving at input (output) nodes).

The tokenising method determines the number of input and output nodes N_I, N_O. Placing $\mathbf{I}$ close to $\mathbf{O}$ can raise the probability hidden layer nodes in $\mathbf{G}$ are visibly close. Yet, manually defining $\mathbf{I}, \mathbf{O}$ could produce sub-optimal arrangements. One could instead let $\mathbf{I}, \mathbf{O}$ freely vary, conditioning their placement on $\mathcal{R}$ for example.

3.4 Fixed $\mathbf{I}, \mathbf{O}$

Fixing $\mathbf{I}, \mathbf{O}$ nodes on curves, surfaces, solids, or in other regular arrangements can ease visualisation. One then optimises the medial[7] node positions relative to $\mathbf{I}, \mathbf{O}$. Updating $\mathbf{G}$ is a simpler operation with $\mathbf{I}, \mathbf{O}$ fixed, not variable.

3.5 Variable $\mathbf{I}, \mathbf{O}$

The input-output node positions could instead vary. Condition optimal $\mathbf{I}, \mathbf{O}$ nodes on medial node positions. Each $\mathbf{I}, \mathbf{O}$ node could occupy an unconstrained number of coordinates, with the $\mathbf{I}, \mathbf{O}$ counts similarly free. A coupled optimisation problem arises:

$$\mathbf{I}_t^* = T_{\mathbf{I}}(\mathbf{G}_t^*) \tag{16}$$

$$\mathbf{O}_t^* = T_{\mathbf{O}}(\mathbf{G}_t^*) \tag{17}$$

$$\mathbf{G}_{t+1}^* = T_{\mathbf{G}}(\mathbf{I}_t^*, \mathbf{O}_t^*) \tag{18}$$

T is the optimising transform. Several best-performing arrangements may exist. Distribute each $\mathbf{I}, \mathbf{O}$ node with a separate random variable:

$$\mathbf{X}^{(i)} \sim f_i \quad \forall i \in \{1, 2, \ldots, N_I + N_O\} \tag{19}$$

In high-dimensional vector embeddings, $N_I + N_O$ is large and thus, parameters specifying all f_i are numerous. Minimise f_i's parameters with simpler distributions reduces optimisation complexity.

[7] The common term "hidden node/layer" is inappropriate given this study's visual emphasis.

4 Realisation

Realisation and optimisation produce a task optimal graph $\mathbf{G}^*$ in $\mathbb{R}^3$. To avoid separately optimising N and $\mathbf{x}^{(i)}$ consider only methods whereby a PDF completely implies the nodes' number and position. Then, to realise $\mathbf{G}^*$ apply a realising rule:

$$f \to \mathbf{G} \tag{20}$$

Useful realisation properties are:

1. $\mathbf{G}$'s realisation from f is stochastic.
2. $\mathbf{G}$ varies with f so optimisation changes $\mathbf{G}$ through f,
3. The number of core node in $\mathbf{G}$ is finite (for implementation),
4. Higher PDF values imply higher realisation frequencies.

Formally, Property (1):

$$\mathcal{M} : \mathbb{F} \times \Omega \to \mathbb{R}^{N(f)\times 3} \tag{21}$$

$$\mathbf{G}(f,\omega) = \begin{pmatrix} x_1^{(1)}(f,\omega) & x_2^{(1)}(f,\omega) & x_3^{(1)}(f,\omega) \\ x_1^{(2)}(f,\omega) & x_2^{(2)}(f,\omega) & x_3^{(2)}(f,\omega) \\ \vdots & \vdots & \vdots \\ x_1^{(N(f))}(f,\omega) & x_2^{(N(f))}(f,\omega) & x_3^{(N(f))}(f,\omega) \end{pmatrix}. \tag{22}$$

and:

$$\forall f \in \mathbb{F}, \exists\, \omega_1 \neq \omega_2 \in \Omega : \mathbf{G}(f,\omega_1) \neq \mathbf{G}(f,\omega_2) \tag{23}$$

Property (2):

$$\exists f_1 \neq f_2 \in \mathbb{F} : \mathbf{G}(f_1,\omega_1) \neq \mathbf{G}(f_2,\omega_1), \tag{24}$$

Property (3):

$$N(f) \in [0,\infty). \tag{25}$$

Property (4):

$$\forall \mathbf{x}', \mathbf{x} \in \mathbb{R}^3,\, f(\mathbf{x}') > f(\mathbf{x}) \implies \lim_{n\to\infty} P(\frac{N(\mathbf{x}',n)}{n} > \frac{N(\mathbf{x},n)}{n}) = 1 \tag{26}$$

where:

- $\mathcal{M}$ is the realisation mapping,
- $\mathbb{F}$ is the function space,

- $N(f)$ is the PDF-dependent node number (maximal row number).
- Ω is a sample space of random states ω.
- $x_d^{(i)}$ are (f, ω)-dependent node coordinates.
- $\frac{N(\mathbf{x},n)}{n}$ is the number of node occurrences at $\mathbf{x}$ in n repeated realisations from f

5 Propagation

Having obtained a PDF and its graph realisation, one must stipulate the network's parameters. Networks containing only multiplicative edge weights, additive node biases, and non-linear activation functions are extremely general [25][26][27]. When highly parameterised and trained with enough data, basic MLP networks are universal function approximators.

Using directed, acyclic computational graphs (DAG) is common in machine learning. Cyclical graphs like RNNs are also well-studied. Directed graphs without cycles are simpler to train because the loss function gradient at each parameter is simpler to calculate, and thus, back-propagation is more readily applied.

A directed, acyclic graph may be inefficient[28]. Consider a neural network with two sub-networks. Sub-network 1 generates conjectures, while sub-network 2 scrutinises conjectures. A DAG cannot send conjectures and improvements between the sub-networks because the information encoding these conjectures cannot cycle within the network. Thus, sub-network 1 must contain many sequential layers s.t. at each layer, a sub-network 2 output can perturb any erroneous conjecture. Thus, rather than two sub-networks, the two networks are elongated across the propagation direction

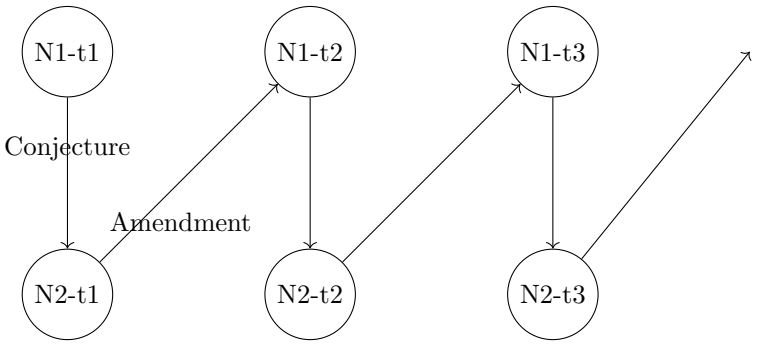


Figure 2: Replicating Sub-networks in DAGs

If information can cycle instead, one reduces the network (figure 3). Reducing the replication in figure 2 is crucial since many neurological processes like introspection, reasoning, and emotion regulation use recursion.

Figure 3 has no terminal time-step, and backpropagated gradients may vanish or explode if the cycle runs indefinitely. This study does not use backpropagated gradients, so calculating gradients is not an issue. Calculating outputs and rewards with possibly infinite recursion does, however, raise the issue of exploding and vanishing activation states. A recursive multiplication operation can quickly cause states to grow large enough or small enough to trigger precision errors during execution. Binary activation functions prevent value errors by pre-

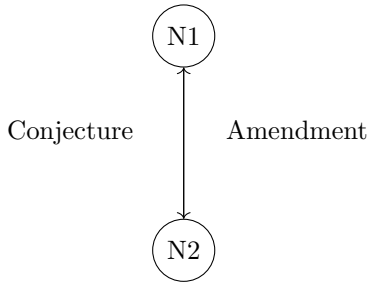


Figure 3: Cyclical Sub-networks

defining feasible propagation values.

If one allows cyclical patterns (bi-directional edges also produce recursion between nodes), then a single input may trigger a cascade of continuous activations. This trait may be useful if the network is to develop thoughts and conjectures independently of external input. One likens these continuous recursions to rumination; activity that persists after removing a stimulus.

Consider 'catching' these recursions at pre-defined time intervals to ensure the network produces some intelligible output. For example, consider the time steps:

$$t = 0, 1, 2, \dots \tag{27}$$

Then specify the receiving time steps τ

$$\tau = \{t_r, t_{r+1}, \dots, t_{r+N}\} \tag{28}$$

If the input enters the network at $t = 0$, the receiving time steps will yield a set of output vectors. The output vectors could be concatenated, averaged, or otherwise combined.[8] Any earlier or later outputs are ignored, and the network can continue to recursively self-signal. The system may, therefore, output long sequences $\mathbf{Y}_1, \mathbf{Y}_2, \dots$ where the output at time t arises due to a prior input $\mathbf{X}_{t-n}$. An output delay necessarily occurs in systems with no direct $\mathbf{I}, \mathbf{O}$ connection.

[8] One need not record outputs contiguously.

6 Update

6.1 Rewards

In the static case:

$$\forall \mathbf{X}, \exists \mathbf{Y}^* \text{ s.t. } \mathbf{Y}^* = \arg\max_{\mathbf{Y} \in \mathbb{Y}} \mathcal{R}(\mathbf{Y}; \mathbf{X}) \tag{29}$$

where:

- $\mathbf{X}$ is the contemporaneous input in the output space $\mathbb{X}$,
- $\mathbf{Y}$ is the contemporaneous output in the output space $\mathbb{Y}$,
- $\mathcal{R}$ is the reward function.

$\mathcal{R}()$ is well-defined over all $\mathbf{Y} \in \mathbb{Y}$ and all $\mathbf{Y}$ are not equally valued. Each $\mathbf{X}$ invokes a different reward profile $\mathcal{R}(\,\cdot\,; \mathbf{X})$. In the dynamic case, rewards $\mathbf{X}_t, \mathbf{Y}_t$ belong to time-series $\mathbf{X}_{\underline{t}:\bar{t}}, \mathbf{Y}_{\underline{t}:\bar{t}}$:

$$\forall \mathbf{X}_{\underline{t}:\bar{t}} : \ \exists \mathbf{Y}^*_{\underline{t}:\bar{t}} = \arg\max_{\mathbb{Y}_t} \mathcal{R}(\mathbf{Y}_{\underline{t}:\bar{t}}; \mathbf{X}_{\underline{t}:\bar{t}}) \tag{30}$$

Where $\mathbb{Y}_t$ is the time-series output space. Static and dynamic rewards are identical, except that static rewards are sequentially independent. In the dynamic case, the total reward $\mathcal{R}$ is a function of weighted static rewards. The network can learn from streamed data to perform real-time tasks during continuous learning. In this study, the learning task is continuous but sequentially independent $\mathbf{I}, \mathbf{O}$ (image classification) rewards are used for simplicity.

6.2 Transform

A reward-dependent update transforms the PDF:

$$f_{t+1} = T(f_t, \mathcal{R}_t) \tag{31}$$

Visually intuitive updates clarify the relation between $\mathbf{X}_{\underline{t}:\bar{t}}, \mathbf{Y}_{\underline{t}:\bar{t}}, \mathcal{R}$ and f. Useful update properties are:

1. $\mathbf{G}_t$'s realisation probability varies positively with performance.

2. The estimator $\hat{f}_t$ converges to the globally optimal function f^* as the sample sequence tends to the population $\mathbb{X}_{\underline{t}:\bar{t}}; \mathbb{Y}_{\underline{t}:\bar{t}}$.
3. If there exists a sample-optimal function f_t^* not the global optimal f^*, T identifies f_t^* given $\mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t}$.
4. Where $f_t^* \neq f^*$, T minimises overfitting.

Formally, Property (1):

$$P(\mathbf{G}) = h(\mathcal{R}; T) \tag{32}$$

$$\exists T : \frac{dP(\mathbf{G})}{d\mathcal{R}} > 0 \tag{33}$$

Property (2):

$$\lim_{\mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t} \to \mathbb{X}_{\underline{t}:\bar{t}}; \mathbb{Y}_{\underline{t}:\bar{t}}} \Delta(\hat{f}_t, f^*) = 0 \tag{34}$$

Property (3):

$$\Delta(\hat{f}_t, f_t^*) = 0 \quad | \quad \mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t} \tag{35}$$

Property (4):

$$\min_{\hat{f}_t}(\Delta(\hat{f}_t, f_t^*) + \lambda \Delta(\hat{f}_t, f^*)) \tag{36}$$

Where:

- $\mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t}$ is a sample data sequence drawn from the population $\mathbb{X}_{\underline{t}:\bar{t}}; \mathbb{Y}_{\underline{t}:\bar{t}}$
- f^* is the globally optimal PDF that maximizes $\mathcal{R}$ on population $\mathbb{X}_{\underline{t}:\bar{t}}; \mathbb{Y}_{\underline{t}:\bar{t}}$.
- $\hat{f}_t$ is the transform's (T) output at t given sample $\mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t}$.
- f_t^* is the sample-optimal PDF maximising $\mathcal{R}$ on $\mathbf{X}_{\underline{t}:t}; \mathbf{Y}_{\underline{t}:t}$.
- $\Delta(f_1, f_2)$ is a distance measure between PDF f_1 and f_2.

Property (1) is highly intuitive, yet the derivative is intractable when $h(.)$ is non-differentiable. Whether the transform reaches f_t^* or f^* as in (2) and (3), avoiding local optima, depends on T's specification (section 7). Property (4) specifies a minimal divergence between the estimated and global optimal mapping. Regularisation can prevent the estimate diverging from the global optimal (avoiding catastrophic forgetting via over-fitting). One could use the Kullback-Leibler divergence between the f_t and f_{t+1} to regularise rewards.

7 Model 1

7.1 Initialisation: Unit Cube

Model 1 uses bounded region intialisation with the unit cube $\mathbf{B}$:

$$\mathbf{B} = \{\mathbf{x} \in \mathbb{R}^3 \mid 0 \leq x_d \leq 1\} \tag{37}$$

Consider a spatial Poisson Point Process (PPP) that generates a cluster of nodes in the unit cube $\mathbf{B}$. The PPP generates N points as a realisation of the random variable $N(l)$:

$$\Pr\{N(l) = n\} = \frac{(\lambda|l|)^n}{n!} e^{-\lambda \cdot |l|} \tag{38}$$

where:

- l is the length of the cube's sides.

One draws node coordinates $\mathbf{x}^{(i)}$ from a uniform distribution over the bounded space in all three dimensions:

$$f(x_d) = \begin{cases} 1 & \text{for } 0 \leq x_d \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Initialise the $\mathbf{I}, \mathbf{O}$ nodes on two parallel square surfaces (S_1, S_2) of a unit cube with vertices:

$$S_1 : (0,0,0), (0,1,0), (0,1,1), (0,0,1) \tag{39}$$

$$S_2 : (1,0,0), (1,1,0), (1,1,1), (1,0,1) \tag{40}$$

Grid-assign the nodes[9]. When $N_I = 784$, $N_O = 1$ as per the image classification task (sec. 8):

$$\mathbf{I} = \begin{bmatrix} 0 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 1 & 1 \end{bmatrix}_{784 \times 3} \qquad \mathbf{O} = \begin{bmatrix} 1 & 0.5 & 0.5 \end{bmatrix}_{1 \times 3} \tag{41}$$

The uniform PDF will deform as an increasing function of $\mathbf{G}$'s performance $r_0 \in \mathbb{R}$ s.t.:

$t = 0$:

$$PDF_0 = U_{(\mathbf{0},\mathbf{1})} \tag{42}$$

At time t:

$$PDF_{t+1} = T(PDF_t) \tag{43}$$

[9] Consider scaling for visibility [24][23]

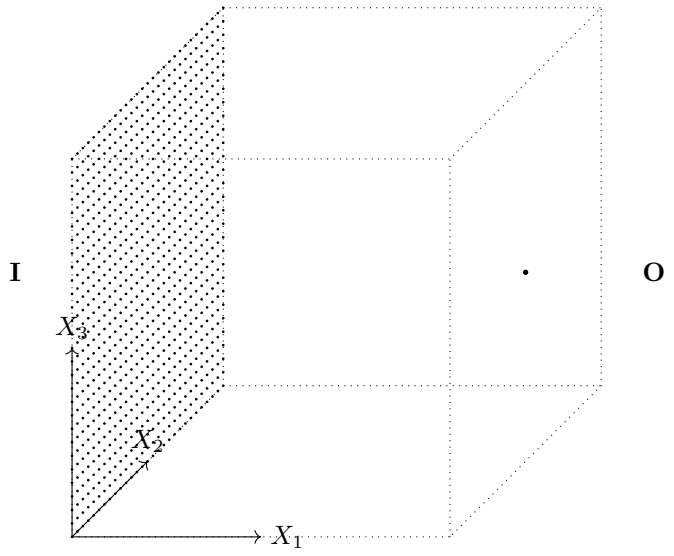


Figure 4: $\mathbf{I}, \mathbf{O}$ nodes at initialisation.

Algorithm 1 Cube Initialisation

Require:
λ: intensity parameter for Poisson distribution
x^{min}, x^{max} ▷ x range
Ensure: Set of points G uniformly distributed in the cuboid
$l \leftarrow x^{max} - x^{min}$ ▷ Cuboid side length
$N \sim \text{Poisson}(\lambda|l|)$ ▷ Number of points to generate
Initialize empty array G
for $i = 1$ to N **do**
    $x_1 \sim \text{Uniform}(x^{min}, x^{max})$
    $x_2 \sim \text{Uniform}(x^{min}, x^{max})$
    $x_3 \sim \text{Uniform}(x^{min}, x^{max})$
    $\mathbf{x}_i \leftarrow (x_1, x_2, x_3)$
    $G \leftarrow [\mathbf{x}_1, ..., \mathbf{x}_{i-1}, \mathbf{x}_i]$
end for
return G

7.2 Realisation: Maximum Object

A piece-wise function normalises f at local maxima:

$$P_{node}(\mathbf{x}') = \begin{cases} \Phi(\mathbf{x}, \mathbf{x}') \cdot f(\mathbf{x}'), & \text{if local maximum} \\ 0, & \text{otherwise} \end{cases} \tag{44}$$

$$\Phi(\mathbf{x}, \mathbf{x}') = e^{-\beta \|\mathbf{x} - \mathbf{x}'\|^2} \tag{45}$$

$f(\mathbf{x}')$ is a local maximum if:

$$\nabla f(\mathbf{x}') = \mathbf{0} \tag{46}$$

$$H_f(\mathbf{x}') \text{ is semi-definite} \tag{47}$$

A node appears at each local maximum with probability P_{node}. If the node always appears exactly at the local maximum, potentially favourable deviations from the maximum go unexplored. So, the node appears according to f's radial basis transformation:

$$\mathbf{x} \sim \Phi(\mathbf{x}, \mathbf{x}') \cdot f(\mathbf{x}) \tag{48}$$

If $f(\mathbf{x})$ is uniformly maximal around $\mathbf{x}'$, the uniformly maximal subset in $\mathbb{R}^3$ is either a curve, a surface, or a solid. A continuous maximum region contains an infinite number of maximal points. To avoid realising a large number of nodes, this study discretises the PDF, thus upwardly bounding the number of feasible maxima. [10]

To realise nodes from the PDF:

1. Use a $3 \times 3 \times 3$ maximum filter on the PDF mesh grid to set the grid's coordinates to the local maximum values,
2. Create a binary mask over the meshgrid that equals 1 if the maximum filter equals the original value at a given point in the mesh,
3. Set mask borders to zero to remove boundary maxima,
4. Define a cube region around each maxima and normalise its densities,
5. Randomly select an element in each local maximum cube according to the contained probabilities.

7.3 Propagation

Model 1 uses a bidirectional (and possibly cyclic) neural network with weighted edges and biasing nodes. The weighted and biased inputs feed into each node's ReLU activation function. Real-valued outputs then propagate outwardly on exiting edges.

[10] One could identify local maxima in proportion to the uniform region's size:

Realise $M \cdot \alpha^d$ nodes in a d dimensional maximum shape.

where:

- Where M is a curve's length, a plane's surface area, or a solid's volume,
- α is a frequency parameter.

Yet, obtaining the region's size and geometry via a level set or derivative method is complex. The current work discretizes space during realisation. Only the grid's maxima can constitute $\mathbf{G}_t$'s nodes, so this study avoids the maximum object problem.

7.3.1 Parameter Conditioning

Despite introducing redundant initial layer parameters, one simplifies notation by applying biases and activation functions to all nodes. Thus:

$$N = N_I + N_O + N(f). \tag{49}$$

$N(f)$ is the number of internal nodes between $\mathbf{I}, \mathbf{O}$. From section 2, the general parameter conditioning relations (5, 6) map $\mathbf{G}$ to $\mathbf{W}, \mathbf{b}$. For node i and its N_i neighbours, the specific parameter-conditioning relations are:

$$w_{ij} = \frac{1}{1 + \mu d_{ij}} \tag{50}$$

$$b_i = \frac{1}{|N_i|} \sum_{j \in N_i} d_{ij} \tag{51}$$

where:

- $d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|$.
- $\mu \in [0, 1]$.

The edge weight between i and j decreases with distance, and i's bias increases with the mean distance to other nodes - a connectivity measure.

Parameter conditioning introduces triangle inequalities between nodes. The weight matrix for N nodes is thus restricted and cannot contain any arbitrary weight combination. Yet, variations in N and $\mathbf{x}^{(i)}$ may facilitate arbitrary *function* approximations.

7.3.2 Operations

Assign an activation state $a_t^{(i)} \in \mathbb{R}$ to each node $\mathbf{x}^{(i)} \in \mathbb{R}^3$. The node state array is:

$$\mathbf{A}_t = \begin{bmatrix} a^{(1)} \\ a^{(2)} \\ \vdots \\ a^{(N)} \end{bmatrix} \in \mathbb{R}^{N \times 1} \tag{52}$$

$$\tag{53}$$

Assign each node $\mathbf{x}^{(i)} \in \mathbf{G}$ a bias and activation operation. Assign each edge a weight $w^{(ij)}$:

$$b^{(i)} \in \mathbb{R}, \tag{54}$$

$$ReLU : max(0, a^{(i)}) \quad or \quad Sigmoid(a^{(i)}) = \begin{cases} 1, & if \quad \sigma(a^{(i)}) \geq 0.5 \\ -1, & otherwise \end{cases} \tag{55}$$

$$w^{(ij)} \in \mathbb{R} \tag{56}$$

Propagation proceeds thus:

1. Given A_t, activate the biased states:

$$Activation(a^{(i)} + b^{(i)}) =^{A} (a^{(i)} + b^{(i)}) \tag{57}$$

$$A_t \leftarrow \begin{bmatrix} {}^{A}a^{(1)} + b^{(1)}) \\ {}^{A}(a^{(2)} + b^{(2)}) \\ \vdots \\ {}^{A}(a^{(N)} + b^{(N)}) \end{bmatrix} \tag{58}$$

2. Given $\mathbf{W}, \mathbf{A}_t$ set the new states as a weighted combination of each node's incoming edges:

$$\mathbf{A}_t \leftarrow \begin{bmatrix} {}^{A}(a^{(1)} + b^{(1)}) \cdot w^{(11)} + \cdots +^{A} (a^{(N)} + b^{(N)}) \cdot w^{(N1)} \\ {}^{A}(a^{(1)} + b^{(1)}) \cdot w^{(12)} + \cdots +^{A} (a^{(N)} + b^{(N)}) \cdot w^{(N2)} \\ \vdots \\ {}^{A}(a^{(1)} + b^{(1)}) \cdot w^{(1N)} + \cdots +^{A} (a^{(N)} + b^{(N)}) \cdot w^{(NN)} \end{bmatrix} = \begin{bmatrix} a'^{(1)} \\ a'^{(2)} \\ \vdots \\ a'^{(N)} \end{bmatrix} \tag{59}$$

The process repeats with the new activation states while the update condition is false.

Algorithm 2 Propagation

```
procedure PROPAGATION(A_t, b_t, W_t)
    while r_t = 0 do                          ▷ Activate biased states
        for i = 1 to N do
            a_t^(i) ←^A (a^(i) + b^(i))
        end for
        for i = 1 to N do                     ▷ Linearly combine the weighted states.
            a_t^(i) ← Σ_{j=1}^{N} a_t^(i) · w^(ij)
        end for
    end while
    return A_t
end procedure
```

7.4 Reward Functions

Model 1 uses a piecewise reward function:

$$R_t(\mathbf{Y}_t; \mathbf{X}_t) = \begin{cases} +1 & , \quad \text{if} \quad \mathbf{Y}_t = \hat{\mathbf{Y}}_t \\ -1 & , \quad \text{if} \quad \mathbf{Y}_t \neq \hat{\mathbf{Y}}_t \\ 0 & , \quad \text{otherwise} \end{cases} \tag{60}$$

This reward function suits the classification task in section 8. Given, the reinforcement-style rewards, the author also trialed a strictly positive continuous reward function:

$$R_t(\mathbf{Y}_t; \mathbf{X}_t) = s_1 \cdot f(\hat{y} - y) \tag{61}$$

where:

1. $f(.)$ may take a piecewise negative quadratic

$$\begin{cases} -(\hat{y} - y)^2 + s_2 & , \quad \text{if} - (\hat{y} - y)^2 + s_2 \geq 0 \\ 0 & , \quad \text{otherwise} \end{cases} \tag{62}$$

2. $f(.)$ may take the Gaussian function $\mathcal{N}\left((\hat{y} - y)|\hat{y}, s_3^2\right)$
3. $s_i \in \mathbb{R}$ is a hyper-parameters.

7.5 Update: Gaussian Mixture

Though realisation uses boundary conditions, node coordinates are not spatially confined during the update process.[11] Consider the Gaussian scaling method:

1. Given $f_t, \mathbf{G}_t, R_t$: Generate one Gaussian at each node $\mathbf{x}^{(i)} \in \mathbf{G}$.

$$\phi_i(\mathbf{x}) = m_i \cdot \frac{1}{\sqrt{(2\pi)^3 \det(\Sigma)}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_i)^T \Sigma^{-1} (\mathbf{x} - \mathbf{x}_i)\right) \tag{63}$$

2. Sum over all Gaussians to obtain a scalar field.

$$\tilde{\Phi} = \Sigma_{i=1}^{N} \phi_i. \tag{64}$$

3. To normalise, integrate over the summed Gaussians.[12]

$$\int_{\mathbb{R}^3} \Phi(\mathbf{x})\, d\mathbf{x} = \sum_{i=1}^{N} m_i \tag{65}$$

4. Divide the summed Gaussians by the integral.

$$\Phi(\mathbf{x}) = \frac{\tilde{\Phi}(\mathbf{x})}{\sum_{i=1}^{N} m_i} \tag{66}$$

5. Use $\Phi(\mathbf{x})$ as a proxy gravitational potential and scale f_t

$$f_{t+1}(\mathbf{x}) = \frac{f_t(\mathbf{x}) + \alpha \cdot R_t \Phi(\mathbf{x})}{1 + \alpha \cdot R_t} \tag{67}$$

α is a parameter that randomly perturbs the updating scalar field's influence on the original function.

[11] Applying boundary conditions to the PDF update is complex and beyond the scope of the current study.
[12] Assuming Σ is the same for all Gaussians, the integral of $\Phi(\mathbf{x})$ over all space simplifies if the Gaussians are not significantly overlapping

8 Experiment

This study evaluates model 1's performance using the Modified National Institute of Standards in Technology (MNIST) database. The MNIST digit classification database includes 70,000 grayscale images (60,000 for training, 10,000 for testing) of handwritten digits (0-9). Each pixel value ranges from 0 to 255, representing grayscale intensity. MNIST's widespread use in classification benchmarking [29][30][31] and simplicity make it an appropriate test for the current study.

The task involved training model 1 on the 60,000 training images. The author then fixed $\mathbf{G}_t$, and tested the graph on the 10,000 test images. In training, a computational graph was initialised using Python, and training images passed sequentially through the model, following the process in section 2. The graph's nodes (and thus parameters) were stored and updated. The author generated a Python matplot figure to visualise the graph and a slice of the current PDF (figure 5).

For time:

$$t \in \mathbb{R}_{\geq 0} \tag{68}$$

1. If $t = 0$, use uniform cube initialisation to define G_t start propagating the model with the first image.
2. If $t \bmod$ (input interval) $= 0$:
   (a) Obtain the current activation vector,
   (b) Check the reward by comparing the output against the current image,
   (c) Perform the reward-dependent update
   (d) Condition weights and biases
   (e) Input $\mathbf{X}_{t+1}$ and return A_{t+1} to the propagator.

An image enters the network at each input interval (e.g., every 3 seconds). Until the subsequent image enters, the correct classification Y_t corresponds to the last input image X_t (e.g. '5' for a handwritten '5'). At the next input run-time, the network received a new image X_{t+1}, and the processes repeated. Both training and testing used this setup. A non-zero reward updated the network using algorithm the algorithm in section 6.

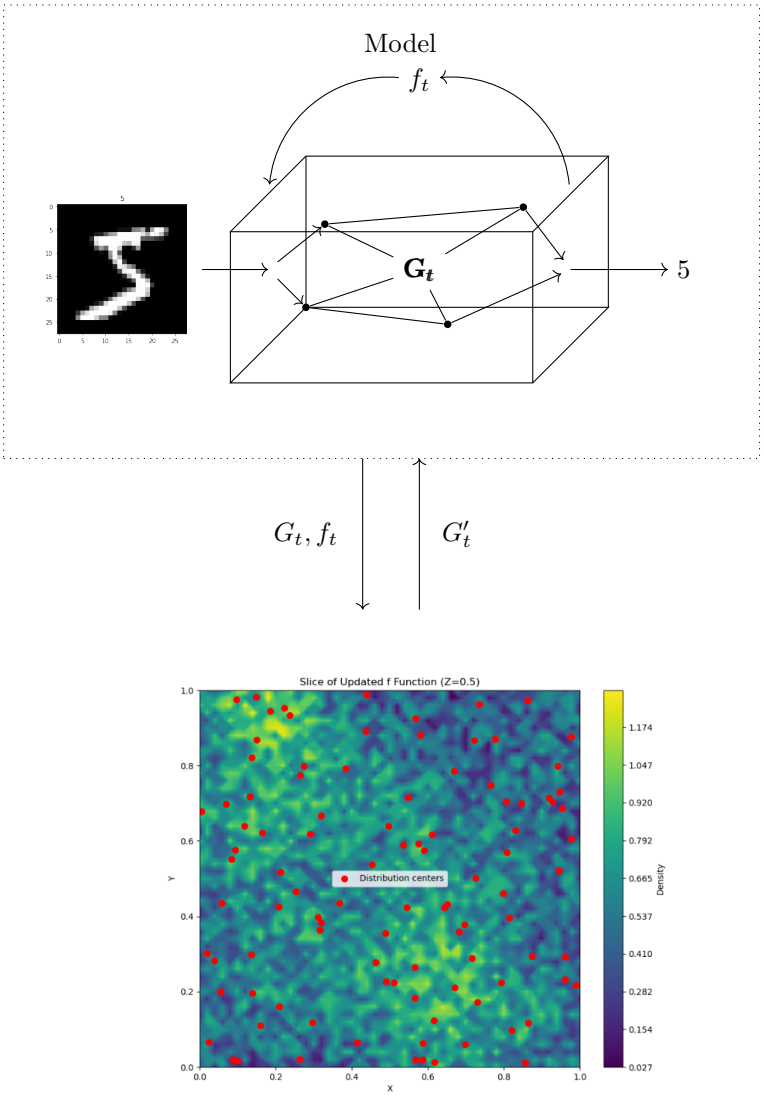


Figure 5: 3D Network Information Flow

9 Performance

Unfortunately, the author could not generate consistent classification results in the permitted time frame. The following results section contains no empirical findings. These results are the project's anticipated findings.

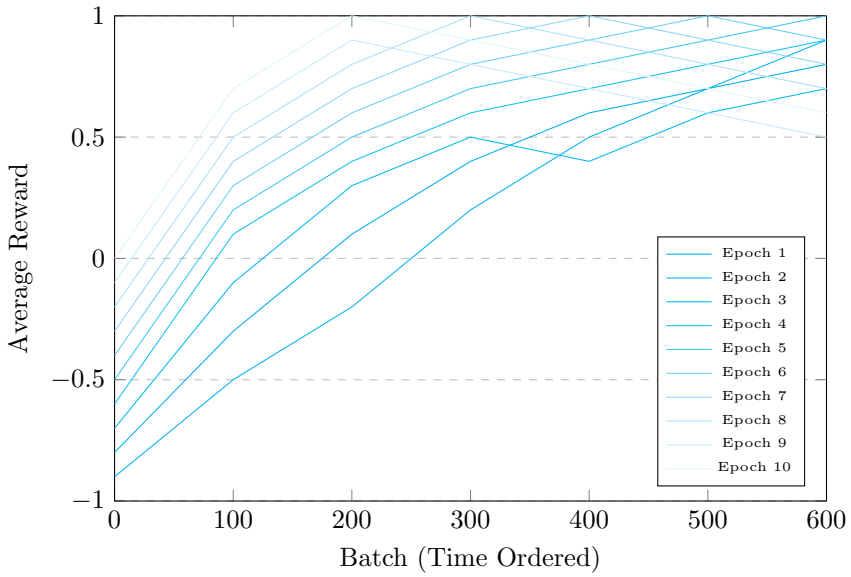


Figure 6: Training: Average Batch Performance

9.1 MNIST Historic Performance

Previous studies have achieved high accuracy (88-99%) on the MNIST database, where human-level accuracy is at least 98%. The MNIST database contains classification study record in the date range 1998-2012 and corresponding test error rates [32]. For example, Ciresan et al. use a CNN with width normalisation to achieve a 0.24% test error [33]. The highest error rate listed on the MNIST database website is 12%, achieved using a simple linear classifier with no preprocessing [34].

9.2 Training Performance

To examine training performance, this study randomly assigns the 60,000 training images into 100 batches of 600 images. The model receives batches sequentially. Once the first 60,000 image epoch finished, the images are randomly re-ordered and randomly re-batched. Then, the model receives the new batches for a second 60,000 image epoch. This process repeats for 10 epochs, and figure 6 displays the performance series for each epoch.

9.3 Test Performance

To examine test performance, this study randomly assigns the 10,000 training images into 100 batches of 100 images. The model receives batches sequentially. The training images are not randomly re-ordered or randomly re-batched. The study tests the 1 training epoch model (1-TrEM), then 2-TrEM, incrementing to 10-TrEM. Figure 7 displays the average batch rewards over time.

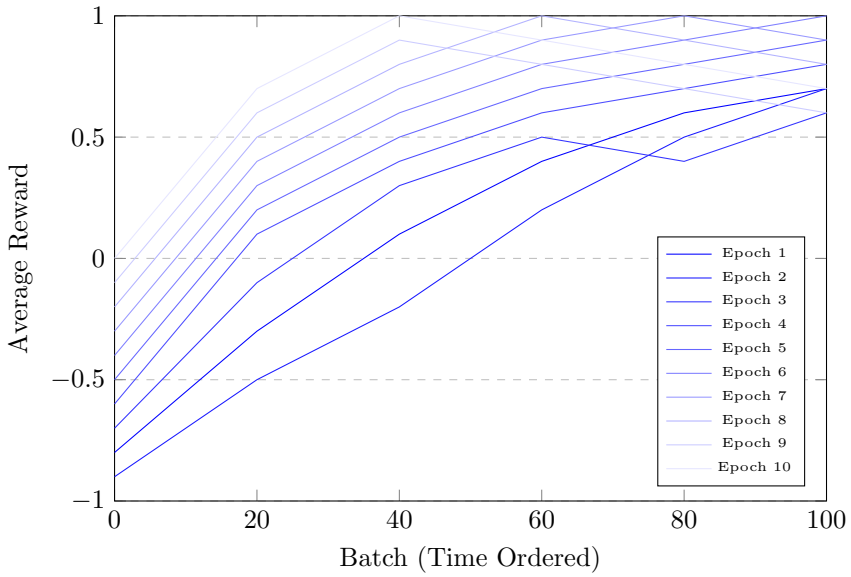


Figure 7: Testing: Average Batch Performance

9.4 Data Augmentation

To examine the model's robustness, the images are augmented, and the model retrained exactly as in section 9.1. Then, each N-TrEM was retested as in section 9.2. The average reward improves significantly as training progresses, with classification errors decreasing over time.

9.4.1 Error at Different Training Orders

The table below shows the classification error rates when varying the order of training images and applying basic image augmentation techniques. These results suggest that image augmentation techniques, particularly when combined, led to modest improvements in the model's generalization capabilities and overall performance.

Training Condition	**Classification Error (%)**
Default order	1.8%
Random shuffle	1.7%
Rotation augmentation (±15°)	1.5%
Translation augmentation (±2px)	1.6%
Combined augmentations	1.4%

Table 1: Classification error rates under different training conditions

10 Geometric Observations

10.1 Structure

Visual inspection reveals the model's structural features. The untrained model (epoch 1, batch 1) has uniformly distributed nodes. As training progresses, the model's vertices begin to cluster and connections become sparser. To quantitatively compare the models' structures, this study calculates the average degree, clustering coefficient, diameter, edge density, and average nearest neighbour values (table 2).

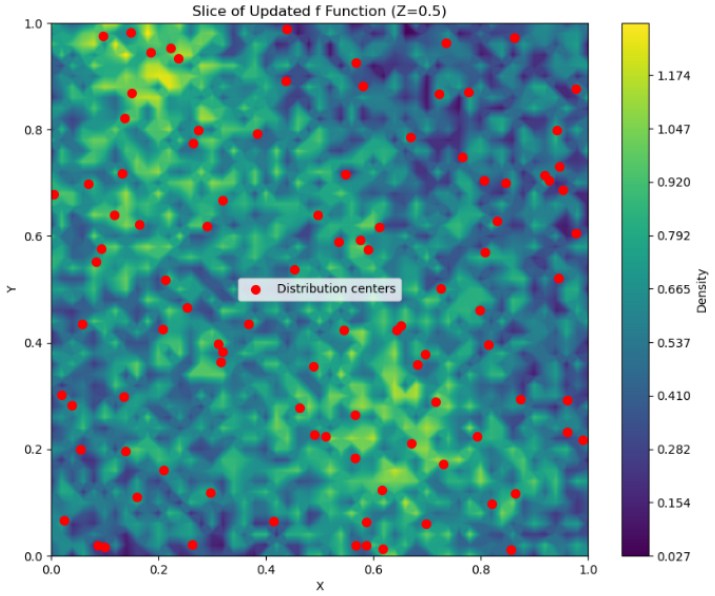
Statistic	**Epoch 1**	**Epoch 2**	**Epoch 10**
Average Degree $\bar{k} = \frac{2M}{N}$			
Clustering Coefficient $C = \frac{3\times\text{triangles}}{\text{connected triples}}$			
Diameter $D = \max d(i,j)$			
Edge Density $\rho = \frac{2M}{N(N-1)}$			
Average Nearest Neighbor Distance $\text{ANN} = \frac{1}{n}\sum_i \min_{j\neq i} d(i,j)$			

Table 2: Post-testing structural features for N-TrEMs

10.2 Graph Structures

10.2.1 Large Scale (radius $\geq$ 0.5)

This study considers structures larger than 0.5 units 'large'. At this scale, nodes occur in clusters of differing densities (Figure 9). Medial nodes arose near input and output nodes with a higher density, graduating to sparser distal arrangements (this clustering actually occurs when running the code in its current form). Point density is highly non-uniform, and nodes form in linked sub-networks when the author made connecting edges stochastic and distance-dependent (also true).

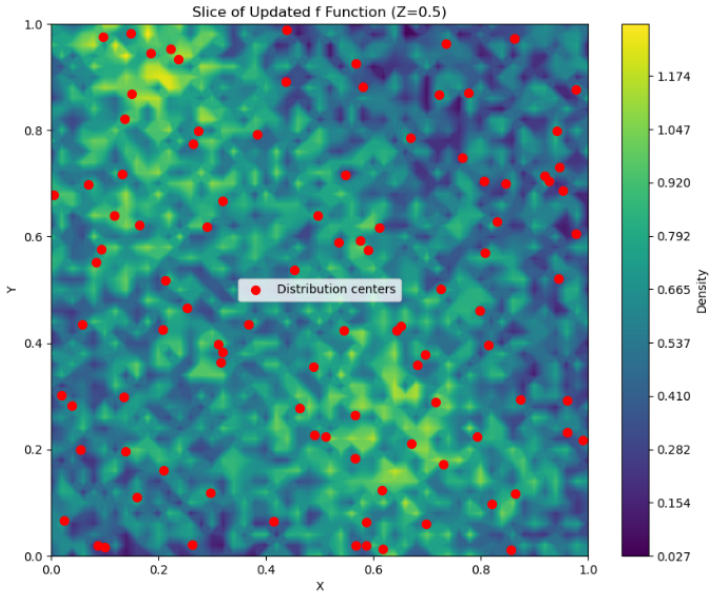


;

Figure 8: 10-TrEM Post-testing Large Structures

10.2.2 Small Scale (radius $<$ 0.5)

This study considers structures smaller than 0.5 units 'small'. The medial nodes clustering around the input and output nodes showed no regular arrangement from above. Medial nodes surrounding the input grid typically had many more connecting edges than medial nodes in other clusters.



;

Figure 9: 10-TrEM Post-testing Small Structures

10.3 Dynamic Features

10.3.1 PDF Deformation

Figure 10 animates the PDF iso-countours for all 10 epochs. These contours store sample-optimal density values. The contours exhibit increasing roughness as training progresses and new medial nodes enter the network.

10.3.2 Activations

Figure 11 displays animates input and medial nodes during testing propagation for the 10-TrEM model. One observes consistent input-dependent activations. One also observes activations in the absence of inputs (as hypothesised in section 5). The consistent relation between input categories and activation patterns mean the network's visual characteristics are highly informative of its function.

These visualizations offer valuable insights into the inner workings of the 3D model, from its overall structure to the behavior of individual components. They demonstrate how the spatial arrangement of nodes and connections contributes to the model's decision-making process, potentially offering new perspectives on neural network interpretability.

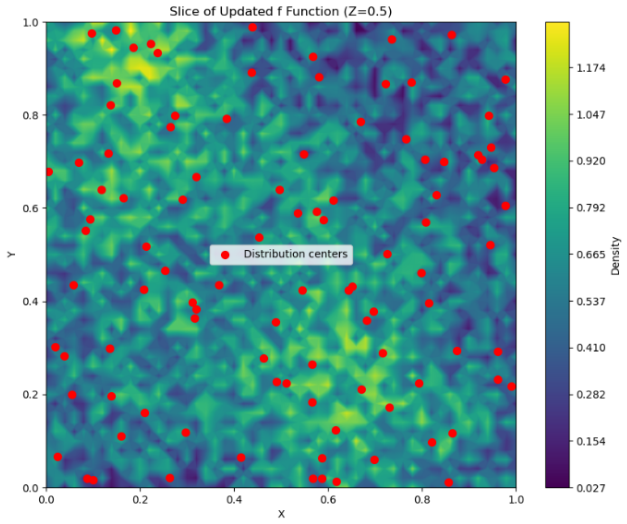


Figure 10: Animation of Iso-contours Across Epochs

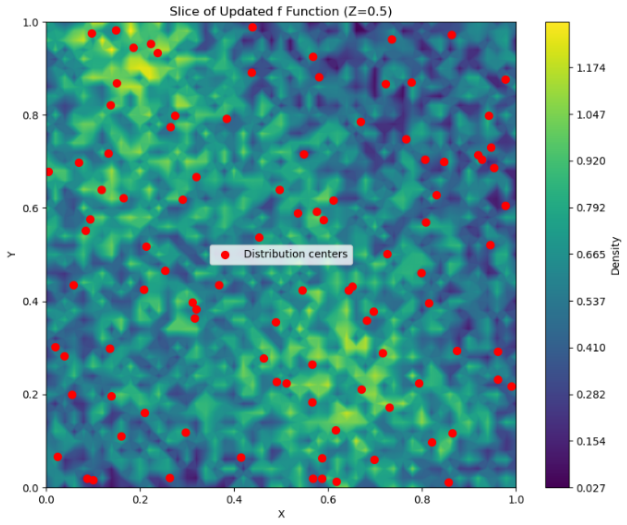


Figure 11: Animation of Node Activations During Testing

10.4 Function

Examining the relation between the model's structure and performance may identify misaligned behaviours before the behaviours arise. Specifically, for true output Y_t a misclassification Y_t' may follow an aberrant activation pattern. To measure activation patterns:

1. Fix the 10-TrEM graph and use the test image set,
2. Given true output Y_t:
3. If the input is correctly classified $\hat{Y}_t = Y_t$:
4. Save the neuron activation values $A \leftarrow \mathbf{X}_t$,
5. Else discard the activation values,
6. Repeat this process for all inputs of class Y_t and record the summed activation values in A,
7. Divide all values in A by the number of correctly classified images.

The average activation graph A now contains the activation profile for correctly classified class members (e.g. handwritten "5"s). The author repeated the above process for misclassified images. This process produced two activation maps A, A'. The difference map $A^d = (A - A')$ was computed and visualised. These maps illustrate functional differences that correlate with misclassification.

10.5 Spherical Ablation

A spherical ablation experiment investigates the model's suspected geometry-performance relation. During the experiment, a uniform distribution produces a point within the unit cube. The point becomes a sphere's centre that deletes all enclosed nodes from the Post-training 10-TrEM model. The test then fixes the deletion and non-deletion architectures and examines both on the 10,000 image test set.

To examine differently scaled subgraphs, the author repeated the test with differing centres and radii. As expected, ablation centres were differently detrimental to performance. Centres within densely clustered nodes reduced performance more than centres within sparse clusters. This result may suggest large graph-spanning edges are unimportant for transforming information.

Surprisingly, the deletion radii did not correlate perfectly with performance reductions. Ablating a few well-connected nodes or clusters greatly reduces performance, whereas ablating many nodes with low connectivity reduces performances minimally.

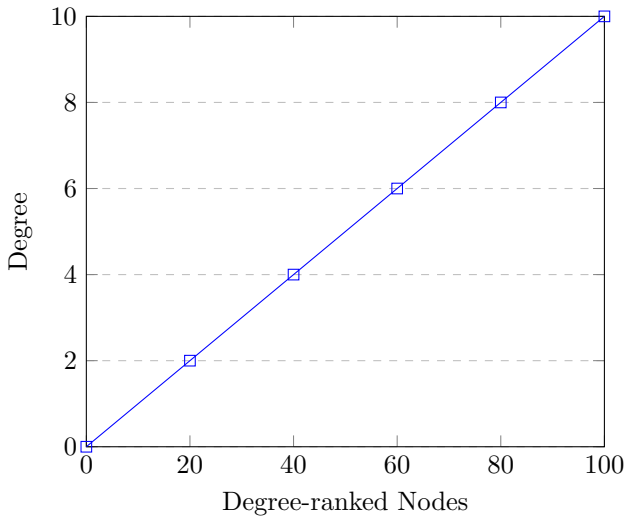


Figure 12: Degree distribution for 1-TrEM Post-training

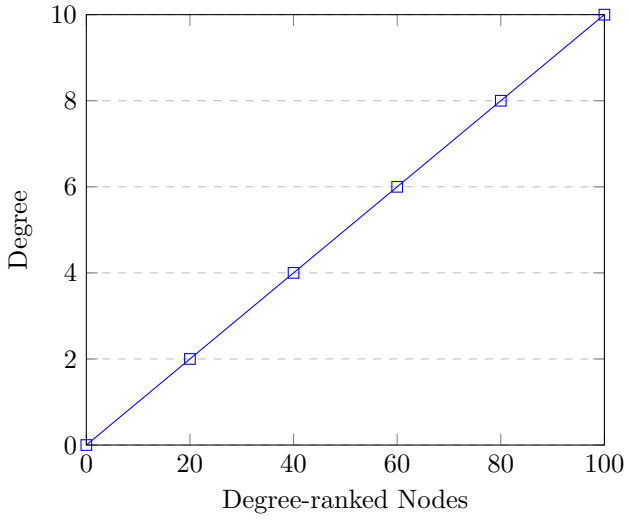


Figure 13: Degree distribution for 10-TrEM Post-training

11 Conclusion

This study introduces a geometric architecture to address interpretability issues in deep learning. 3D networks leverage parameter conditioning to tie the network's function to its geometric structure. This approach illustrates the network's behaviour and draws similarities with imaging techniques used in biological sciences. Conditioning constrains feasible parameter combinations, and an important unanswered question is whether universal function approximation properties extend to conditioned geometric models.

An image classification experiment sought to demonstrate that 3D networks perform accurately on MNIST image classification. Ablation and function mapping were promising avenues for examining the relation between activation patterns and outputs. Activation mapping is a promising avenue for predicting failure or misalignment on more complex tasks, especially when logical proofs are unavailable or too complex for human comprehension.

Unfortunately, despite extensive efforts, the proposed machine learning model failed to produce reliable results due to the inherent sensitivity in the density function used for realisation. Small perturbations in the PDF led to significant variations in the graph's realisation, undermining the method's stability and convergence. Consequently, the model's performance remained inconsistent, preventing the generation of conclusive results within the given timeframe.

Nonetheless, with further refinements, the model could produce accurate empirical results, so opportunities for further research exist. Future work could improve the proposed methods by ensuring the PDF updates relate more reliably to the model's function. Partial differential equations might simplify the PDF specification during the update, thus removing the need for mesh grid discretising (and the associated computations). Additionally, a deeper investigation into the activation patterns might reveal functional relationships (like specialised cortices) not identified in this study. Finally, future work could examine the relevance of 3D networks for human brain imaging as increasingly sophisticated virtual models become reliable analogues for in vivo neuroanatomy.

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[2] L. Alzubaidi, J. Zhang, A. J. Humaidi, *et al.*, "Review of deep learning: Concepts, cnn architectures, challenges, applications, future directions," *Journal of big Data*, vol. 8, pp. 1–74, 2021.

[3] A. Shrestha and A. Mahmood, "Review of deep learning algorithms and architectures," *IEEE access*, vol. 7, pp. 53 040–53 065, 2019.

[4] M. A. Ganaie, M. Hu, A. Malik, M. Tanveer, and P. Suganthan, "Ensemble deep learning: A review," *Engineering Applications of Artificial Intelligence*, vol. 115, p. 105 151, 2022.

[5] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature machine intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[6] A. P. Badia, B. Piot, S. Kapturowski, *et al.*, "Agent57: Outperforming the atari human benchmark," in *International conference on machine learning*, PMLR, 2020, pp. 507–517.

[7] S. Bubeck, V. Chandrasekaran, R. Eldan, *et al.*, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint arXiv:2303.12712*, 2023.

[8] X. Li, H. Xiong, X. Li, *et al.*, "Interpretable deep learning: Interpretation, interpretability, trustworthiness, and beyond," *Knowledge and Information Systems*, vol. 64, no. 12, pp. 3197–3234, 2022.

[9] N. Carlini, F. Tramer, E. Wallace, *et al.*, "Extracting training data from large language models," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.

[10] W. X. Zhao, K. Zhou, J. Li, *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[11] J. Kaplan, S. McCandlish, T. Henighan, *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[12] J. Wei, Y. Tay, R. Bommasani, *et al.*, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022.

[13] N. Shazeer, A. Mirhoseini, K. Maziarz, *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017.

[14] D. Patterson, J. Gonzalez, Q. Le, *et al.*, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[15] E. Strubell, A. Ganesh, and A. McCallum, "The cost of training nlp models: A concise overview," *arXiv preprint arXiv:2004.08900*, 2020.

[16] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, "The computational limits of deep learning," *arXiv preprint arXiv:2007.05558*, 2020.

[17] R. M. Neal, *Bayesian learning for neural networks*. Springer Science & Business Media, 2012, vol. 118.

[18] D. J. MacKay, "Probable networks and plausible predictions-a review of practical bayesian methods for supervised neural networks," *Network: computation in neural systems*, vol. 6, no. 3, p. 469, 1995.

[19] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural network," in *International conference on machine learning*, PMLR, 2015, pp. 1613–1622.

[20] M. P. Hobson, G. P. Efstathiou, and A. N. Lasenby, *General relativity: an introduction for physicists*. Cambridge University Press, 2006.

[21] S. M. Carroll, *Spacetime and geometry*. Cambridge University Press, 2019.

[22] A. Einstein *et al.*, "The foundation of the general theory of relativity," *Annalen Phys*, vol. 49, no. 7, pp. 769–822, 1916.

[23] T. Akenine-Moller, E. Haines, and N. Hoffman, *Real-time rendering*. AK Peters/crc Press, 2019.

[24] J. D. Foley, *Computer graphics: principles and practice*. Addison-Wesley Professional, 1996, vol. 12110.

[25] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, vol. 2, no. 4, pp. 303–314, 1989.

[26] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural networks*, vol. 4, no. 2, pp. 251–257, 1991.

[27] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, "The expressive power of neural networks: A view from the width," *Advances in neural information processing systems*, vol. 30, 2017.

[28] Y. Bai, H. Wang, X. Ma, Y. Zhang, Z. Tao, and Y. Fu, "Parameter-efficient masking networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 10 217–10 229, 2022.

[29] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791.

[30] D. Cireșan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2012, pp. 3642–3649. DOI: 10.1109/CVPR.2012.6248110.

[31] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *7th International Conference on Document Analysis and Recognition*, IEEE, vol. 3, 2003, pp. 958–962.

[32] Y. LeCun, C. Cortes, and C. J. Burges. "The mnist database of handwritten digits." Accessed: 2024-11-14. (1998), [Online]. Available: http://yann.lecun.com/exdb/mnist/.

[33] D. Ciregan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in *2012 IEEE conference on computer vision and pattern recognition*, IEEE, 2012, pp. 3642–3649.

[34] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[35] M. R. Mercier, A.-S. Dubarry, F. Tadel, *et al.*, "Advances in human intracranial electroencephalography research, guidelines and good practices," *Neuroimage*, vol. 260, p. 119 438, 2022.

[36] R. J. Huster, S. Debener, T. Eichele, and C. S. Herrmann, "Methods for simultaneous eeg-fmri: An introductory review," *Journal of Neuroscience*, vol. 32, no. 18, pp. 6053–6060, 2012.

[37] R. Cabeza and A. Kingstone, *Handbook of functional neuroimaging of cognition*. Mit Press, 2006.

[38] S. Liu, D. Maljovec, B. Wang, P.-T. Bremer, and V. Pascucci, "Visualizing high-dimensional data: Advances in the past decade," *IEEE transactions on visualization and computer graphics*, vol. 23, no. 3, pp. 1249–1268, 2016.

Appendices

A Github Address

The project's code files are available at: https://github.com/pascaltohouri/thesis

B Dimensionality

A Euclidean 1-Space

The simplest non-point Euclidean space is the real number line $\mathbb{R}$. In Figure 1, $\mathbf{G}$ is fully connected, and each parameter is a function of node coordinates:

$$w^{(ij)} = \frac{1}{|x^{(i)} - x^{(j)}|} \tag{69}$$

$$b^{(ij)} = e^{-|x^{(i)} - x^{(j)}|} \tag{70}$$

The relations here are trivially defined but demonstrate spatially dependent parameters. Distal nodes tend not to interact, proximal nodes transform information and activate one another more frequently. Drawing the nodes from a 1D PDF produces a linear graph:

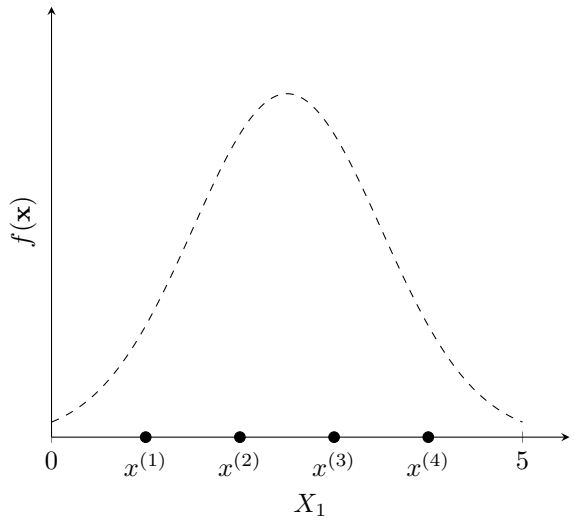


Figure 14: A 1D Gaussian and its network realisation

As in the perceptron, numbers propagate from input node $x^{(1)}$ to output node $x^{(4)}$. The super-imposed edges are functionally independent of one another[13], so information propagates

[13]overlapping edges should arguable interact, but these interactions are increasingly complex in higher dimensional embeddings.

in parallel. The hypothesised node coordinates in figure 1 are:

$$G = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \tag{71}$$

For simplicity, edges from $x^{(1)}$ and to $x^{(4)}$ are uni-directional and $\mathbf{G}$ contains no loops. Self-looping edge parameters are undefined[14]. Weight and bias matrices are thus:

$$W = \begin{bmatrix} - & 1 & \frac{1}{2} & \frac{1}{3} \\ - & - & 1 & \frac{1}{2} \\ - & 1 & - & 1 \\ - & - & - & - \end{bmatrix} \tag{72}$$

$$B = \begin{bmatrix} - & e^{-1} & e^{-2} & e^{-3} \\ - & - & e^{-1} & e^{-2} \\ - & e^{-1} & - & e^{-1} \\ - & - & - & - \end{bmatrix} \tag{73}$$

Again, these parameters are merely illustrative. To declutter the graph and condition parameters on a richer set of spatial information, one must consider higher dimensional spaces.

B Euclidean 2-Space

The 2D Euclidean geometry is the flat plane $\mathbb{R}^2$. The added dimension offers a richer set of information for parameter conditioning. Given node coordinates $\mathbf{x} = [x_1, x_2]^{T(i)}$, let:

$$w^{(ij)} = ||\mathbf{x}^{(i)} - \mathbf{x}^{(j)}|| \tag{74}$$

$$b^{(i)} = \frac{1}{||\mathbf{x}^{(i)}||^2} \tag{75}$$

Where $||\mathbf{x}^{(i)}||$ is $x^{(i)}$'s 2-norm. Again, intersecting edges function independently, and parameter relations are trivial. One can now condition biases and weights on separate dimensions; this added degree of freedom increases the network's expressive ability. One samples nodes from a 2D PDF[15]:

[14] Relaxing this assumption allows repeated activation that facilitates memory.
[15] The node coordinates are regular for simplicity.

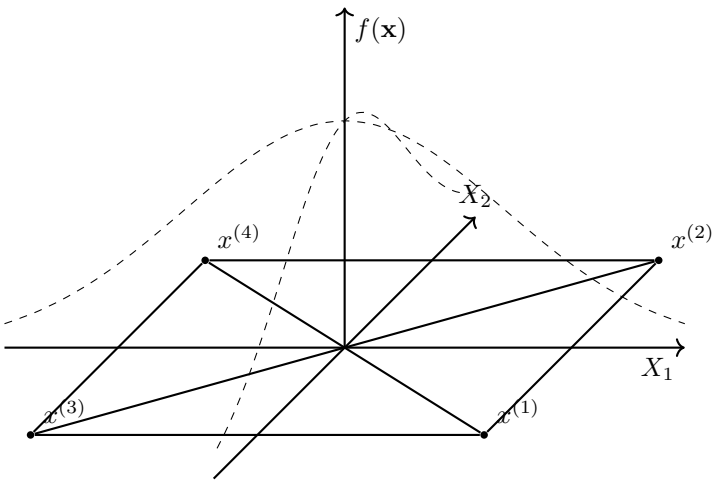


Figure 15: A 2D standardised Gaussian and its network realisation

With:

$$\mathbf{G} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 1 \\ -1 & -1 \end{bmatrix} \tag{76}$$

$$\mathbf{W} = \begin{bmatrix} - & 2 & 2 & 2\sqrt{2} \\ - & - & 2\sqrt{2} & 2 \\ - & 2\sqrt{2} & - & 2 \\ - & - & - & - \end{bmatrix} B = \begin{bmatrix} \sqrt{2} \\ \sqrt{2} \\ \sqrt{2} \\ \sqrt{2} \end{bmatrix} \tag{77}$$

In 2D space, the added degree of freedom allows the network to display more complex representations than on a line. Yet, $\mathbb{R}^2$ is too simple and visually unintuitive to support large networks; the network's structure becomes occluded when node connectivity is high. One must consider higher dimensional spaces.

C Euclidean 3-Space

The Euclidean 3-space mimics humans' perceived physical environment (x_1, x_2, x_3). 3D networks are spatially intuitive and avoid 2D occlusions. The 3-space also offers a richer set of information for parameter conditioning. For node coordinates $[x1, x2, x3]^{(i)}$, let:

$$w_{ij} = \frac{1}{\sqrt{(x_1^{(i)} - x_1^{(j)})^2 + (x_2^{(i)} - x_2^{(j)})^2}} \tag{78}$$

$$b_{ij} = \frac{1}{(x_3^{(i)} - x_3^{(j)})^2} \tag{79}$$

Intersecting edges function independently, and parameter relations are trivially defined. The 3D network can learn more complex representations than the 2D network. One visualises the network like any 3D object. Humans experience biological neural networks in 3D and many of the brain-imaging techniques [35][36][37] developed for human brains apply to 3D neural networks .

D Higher n-Spaces

Geometric networks are visually intuitive and reduce PDF optimisation complexity with parameter conditioning; these are crucial advantages. Adding dimensions increases the PDF's optimisation complexity and increases the stored coordinates for each node. Humans also struggle to visualise higher dimensions that lie beyond their sensory experience [38]. The added mathematical complexity of higher n-spaces outweighs the expressive ability gained in higher dimensions.